Ya sure, I can make a presentation: Computer literacy for physics

UCC Physics Society

[Click to add subtitle]

Learning outcomes

1. Use the terminal and shell or whatever.
2. Create a project and understand a file path and folders.
3. Install and run python and pip.
4. Create Python environment for the project for your project.
5. Use Git/GitHub to save project history.
6. Use LateX either in vscode/codium, overleaf or another compiler.
7. Install the Linux Terminal.

Different Operating Systems

- Windows: PowerShell, Python, Git, and an editor.
- macOS: Terminal/zsh, Python, Git, and an editor.
- Debian based: terminal, Python, Git, and an editor.
- LaTeX: Overleaf in a browser or a local TeX distribution.

Using the terminal/shell and understanding file paths

$$\text{Terminal} \longrightarrow \text{User} \longrightarrow \text{Folders} \longrightarrow \text{Files}$$

- The terminal allows you to interact with the shell and navigate the file system.
- A file system is a hierarchical structure of directories (folders) and files.
- Through the terminal you can run commands and open files.
- You are able to install python and run pip commands to create things.
- There are many other things you can do through the terminal like running code and ssh into remote servers, that will not be covered here.

File paths and directories

```
.       current directory
..      parent directory
```

- Relative paths start at the process's working directory.
- File names are case-sensitive and please dont put spaces in them it makes it difficult to specify paths.
- Use forward slashes '/' as directory separators for better compatibility across different operating systems.
- cd is the command to "change directory".

Virtual environment for Python project

- Create and activate a project environment before installing packages.
- Install packages with pip, then use `deactivate` when finished.

Linux

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install numpy
python3 -m pip install -r
    requirements.txt
deactivate
```

macOS

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install numpy
python3 -m pip install -r
    requirements.txt
deactivate
```

Windows

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install numpy
python -m pip install -r
    requirements.txt
deactivate
```

for windows you may have to run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

LaTeX: describe structure, then build

LaTeX is a typesetting system for producing scientific and technical documents.

- `.tex`: report source.
- `.bib`: bibliographic records.
- `.pdf`: generated document.

$$\int_{-\infty}^{\infty} \frac{\phi^* \times \psi}{\sqrt{\text{bread}}} \, \mathrm{d}r = \text{oodles of dough} \times \langle \phi | \psi \rangle = 1 \text{ or something}$$

$$\sigma_f^2 \simeq \sum_i \left(\frac{\partial f}{\partial q_i} \right)^2 \sigma_{q_i}^2.$$

Column 1	Column 2	Column 3
Entry 1	Entry 2	Entry 3
Entry 4	Entry 5	Entry 6

Structure of a project/LabReport

```
pendulum-lab/
    README.md              # this is a brief overview of the project
    requirements.txt
    data/                  # this folder contains all the data files
        raw_measurements.txt
        raw_measurements_processed.csv
    analysis/              # code for making graphs and analysis
        analysis_script.py
        analysis_script.ipynb
    figures/               # neat way of organizing figures
        Graph.png
        Graph.jpg
    report/                # this folder contains the final report
        report.tex
        report.pdf
```

Git

Git: local history

Commits, branches.
Works offline.
Records snapshots.

GitHub: online collaboration

Hosts repositories.
Collaborates online.
Is not local .

Save $\neq$ commit $\neq$ push.
Save is local to your working directory; Commit records a snapshot in the repository; Push updates the remote repository.

Habits to learn

1. Preserve raw data, units, and provenance.
2. Record environment and run instructions.
3. Make small commits with meaningful messages.
4. Keep independent backups and test restoring them.
5. get enough sleep.
6. make shortcuts for frequent tasks.
7. keep things organized in folders, with clear naming conventions.
8. switch to linux.
9. regularly update all of your packages.
10. Learn to trouble shoot code yourself

You do not need to memorise every command

- Shortcuts can be made with a custom keybinding or script. - easier on Linux.
- Use the built-in help and documentation for commands and functions.
- Git can be used inside vscode without the terminal.
- This talk comes with a github repository that walks through most things needed or covered in this workshop but in more detail.

Open the workshop reference material on GitHub

Useful References

- Pro Git
- GitHub documentation
- Overleaf learning resources
- Python virtual environments
- Matplotlib user guide
- SciPy documentation